

CCE1 EXAMINATION QUESTION BANK — MASTER MODEL THEORY SOLUTIONS

Course Pattern: TE IT - 2024 Pattern (SPPU) **Academic Term:** Semester V (2025-26 / 2026-27)
Document Type: In-Depth SPPU Theory Exam Model **Course Code:** PCC-302-IT / PCC-302-ITT
Answers

IMPORTANT:

SPPU Model Theory Answer Standard: This master compilation provides comprehensive, exam-ready answers formulated specifically to satisfy SPPU 7-mark and 8-mark evaluation schemes. All 14 questions from the official CCE1 Question Bank across Unit 1 and Unit 2 are consolidated into **9 exhaustive topic solutions** featuring point-wise architectural breakdowns, parameter-based comparative tables, labeled system diagrams, PEAS specifications, formal 5-tuple problem formulations, search tree traces, and deductive logic inference sequences.

Unit 1: Introduction to Artificial Intelligence and Intelligent Agents (CO302.1)

Topic 1.1: Artificial Intelligence Definition, Four Quadrants & Types of AI

GROUP:

Consolidated Questions Covered:

- **Q1:** What do you mean by Artificial Intelligence (AI)? What are the different types of AI? Explain any one type of AI in detail. (BTL 2, 7 Marks)

1. Academic Definition of Artificial Intelligence: **Artificial Intelligence (AI)** is a branch of computer science concerned with building smart computational systems capable of performing tasks that typically require human intelligence, such as visual perception, natural language understanding, logical reasoning, decision-making, and autonomous learning from experience.

In standard academic literature (**Russell & Norvig**), AI definitions are organized into a **Two-Dimensional Matrix (Four Quadrants)** along two primary axes: **Thought Process vs. Behavior**, and **Human Performance vs. Rationality (Ideal Performance)**:

Dimension	Human-Centric Approach	Rationalist / Ideal Approach
Reasoning & Thought	Thinking Humanly: Cognitive Science approach; modeling mental thought processes through neural simulations and cognitive models of human memory.	Thinking Rationally: The “Laws of Thought” approach based on formal logic, syllogisms, and provable deductive reasoning.
Behavior & Action	Acting Humanly: The Turing Test approach; building machines that act indistinguishably from a human in communication and perception.	Acting Rationally: The Intelligent Agent approach; designing rational agents that act to achieve the best expected outcome given available percepts.

2. Classification / Types of Artificial Intelligence: AI is categorized into three primary evolutionary tiers based on capability and scope of intelligence:

AI Capability Tier	Core Concept & Scope	Real-World Examples
1. Narrow AI (ANI) (Weak AI)	Specialized AI trained to excel at a single specific, dedicated task without generalizing outside its domain.	AlphaGo, Apple Siri, Tesla Autopilot, Netflix Recommendation Engine, ChatGPT (LLMs).
2. General AI (AGI) (Strong AI)	Hypothetical human-level intelligence capable of abstract reasoning, cross-domain knowledge transfer, and emotional adaptability across all cognitive tasks.	Theoretical research goal; no production AGI system currently exists in industry.
3. Super AI (ASI) (Superintelligence)	Hypothetical stage where machine intelligence surpasses cumulative human intellect across creativity, scientific discovery, and social skills.	Theoretical future stage post-technological singularity.

3. In-Depth Technical Analysis of Narrow AI (Artificial Narrow Intelligence - ANI): Since Narrow AI constitutes 100% of modern operational AI implementations, its architectural details include:

- **Operational Mechanism:** Relies on statistical machine learning models (Deep Neural Networks, Transformers, CNNs, SVMs) trained on vast domain-specific datasets. It maps inputs X to outputs Y ($f: X \rightarrow Y$) through mathematical optimization of loss functions.
- **Key Characteristics:**
 1. **High Domain Precision:** Outperforms human experts in narrow domains (e.g., DeepMind's AlphaFold predicting 3D protein structures with sub-angstrom accuracy; medical image classification detecting malignant melanomas).
 2. **Lack of Common Sense & Brittleness:** Incapable of contextual common-sense reasoning; easily fooled by adversarial perturbations or out-of-distribution data.
 3. **Inability to Transfer Knowledge:** A model trained for chest X-ray diagnosis cannot interpret conversational speech or play chess without retraining from scratch.
- **Architecture Pipeline:** Data Collection & Annotation → Feature Extraction → Model Training (Gradient Descent & Backpropagation) → Hyperparameter Tuning → Inference Deployment.

Source: Russell & Norvig (AIMA 4th ed), Ch 1, p. 1-28 | Source: Alpaydin, Ch 1, p. 1-15

Topic 1.2: Intelligent Agents, Utility-Based Agent Architecture & Comparison with Model-Based Reflex Agents

GROUP:

Consolidated Questions Covered:

- **Q2:** Define an Intelligent Agent. Explain the Utility-Based Agent in detail with a suitable example. Compare a Utility-Based Agent with a Model-Based Reflex Agent. (BTL 3, 7 Marks)
- **Q3:** Apply the concept of a Utility-Based Agent to a suitable real-world scenario and compare its decision-making process with that of a Model-Based Reflex Agent. (BTL 4, 8 Marks)

1. Academic Definition of an Intelligent Agent: An **Intelligent Agent** is an autonomous entity that perceives its external environment through **Sensors**, processes the received percept history via an **Agent Program**, and acts upon the environment through **Actuators** to achieve a specific set of goals rationally.

Mathematical Formalization:

- Let P denote the set of all possible individual percepts.
- Let P^* denote the **percept sequence** (complete history of everything the agent has perceived up to current time t).
- Let A denote the set of possible actions.
- The **Agent Function** is an abstract mathematical mapping:

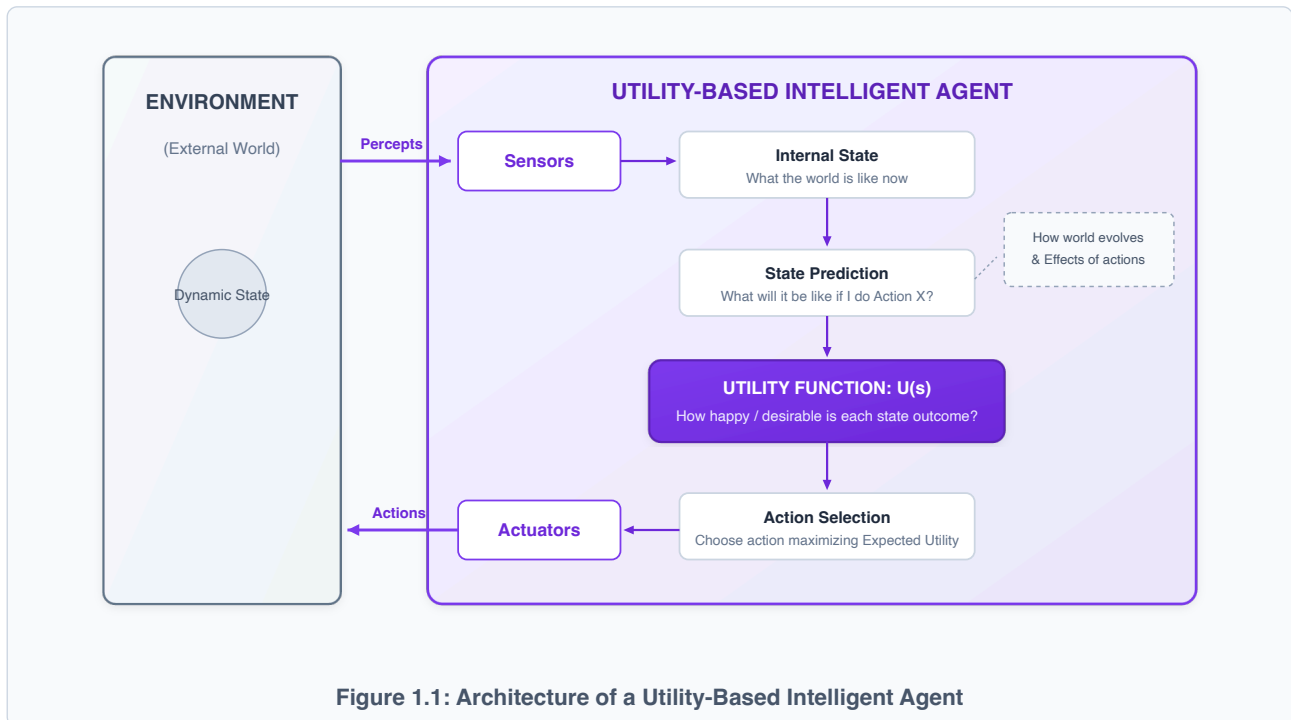
$$f: P^* \rightarrow A$$

- The **Agent Program** is the concrete physical algorithm executing inside an architectural computing platform that implements the agent function f .

2. Utility-Based Agent Architecture & Working Principle: While Goal-Based Agents make crude binary distinctions between “goal states” and “non-goal states” (success vs. failure), real-world multi-objective environments require evaluating **trade-offs** (e.g., speed vs. safety vs. fuel efficiency).

A **Utility-Based Agent** incorporates an explicit mathematical **Utility Function** $U(s)$ that maps an environmental state s (or sequence of states) onto a real number representing the agent’s degree of preference or “happiness”:

$$U : S \rightarrow \mathbb{R}$$



Step-by-Step Decision-Making Cycle of a Utility-Based Agent:

1. **Percept Processing:** Sensors capture raw data from the environment and update the agent’s internal **State** (maintaining a model of how the world evolves independently and what its own actions achieve).
2. **State Projection & Outcome Simulation:** The agent projects alternative candidate actions $a_1, a_2, \dots, a_k$ and uses its transition model $P(s' | s, a)$ to simulate the resulting future states s' .
3. **Utility Evaluation:** The utility function $U(s')$ scores each simulated outcome based on competing criteria (e.g., travel time, passenger comfort, collision risk, toll expenses).
4. **Expected Utility Maximization:** Under environmental uncertainty, the agent selects the action a^* that maximizes **Expected Utility (MEU Principle)**:

$$a^* = \operatorname{argmax}_a \sum_{s'} P(s' | s, a) \cdot U(s')$$

5. **Actuator Execution:** The chosen optimal action is dispatched to the actuators.

3. Real-World Application Scenario: Autonomous Ride-Hailing Navigation Agent:

- **Scenario:** An autonomous taxi at Point A needs to transport a passenger to the Airport (Point B) during peak morning traffic.
- **Utility-Based Agent Decision:**
 - **Route 1 (Highway):** Fast (25 mins), High Toll Cost (\$15), High Speed, Low Congestion risk.
 - **Route 2 (City Arterial):** Moderate (35 mins), Zero Toll, Heavy braking/stops, Moderate Congestion risk.
 - **Route 3 (Backstreets):** Slow (45 mins), Zero Toll, Scenic, Zero Congestion.
 - **Utility Function Formulation:** $U(s) = w_1 \cdot (-\text{Time}) + w_2 \cdot (-\text{Cost}) + w_3 \cdot (\text{Comfort}) + w_4 \cdot (\text{Safety})$.
 - **Decision:** Computes the weighted scalar utility score and dynamically selects Route 1 when the passenger has a tight flight departure, or Route 2 when passenger prefers cost minimization.

4. Parameter-Based Comparison: Utility-Based Agent vs. Model-Based Reflex Agent:

Parameter	Model-Based Reflex Agent	Utility-Based Agent
Internal Knowledge	Maintains an internal model of the world to track partially observable states.	Maintains world model plus an explicit mathematical utility function $U(s)$.
Decision Mechanism	Rule-driven: Uses Condition-Action rules (IF state THEN action) based on current state.	Optimization-driven: Evaluates simulated future states and selects action maximizing expected utility.
Handling Trade-Offs	Cannot handle conflicting goals (e.g., speed vs. safety); executes fixed heuristic rules.	Excels at multi-objective optimization and balancing quantitative trade-offs.
Adaptability under Uncertainty	Rigid; fails if unexpected state has no matching condition-action rule.	Robust; calculates probabilistic expected utility ($\sum P(s' s, a)U(s')$) under uncertainty.
Computational Complexity	Low; simple table/rule lookup after state update ($O(1)$ to $O(R)$).	High; requires state space projection, probabilistic calculations, and optimization search.
Goal Representation	Implicitly embedded inside condition-action rules.	Explicitly parameterized as a continuous scalar utility score ($U : S \rightarrow \mathbb{R}$).
Automated Driving Example	If Brake_Lights_Ahead is detected in internal state, immediately execute Apply_Brakes rule.	Evaluates braking vs. lane-changing based on passenger comfort, arrival time, and collision risk scores.

Source: Russell & Norvig (AIMA 4th ed), Ch 2, p. 36-62

Topic 1.3: PEAS Framework & Agent Environment Specification

GROUP:

Consolidated Questions Covered:

- **Q4:** Explain the PEAS framework used for specifying an intelligent agent. Illustrate PEAS with a suitable example of an autonomous vehicle system or medical diagnosis system. (BTL 3, 7 Marks)
- **Q6:** Apply the PEAS framework to specify an intelligent agent for an autonomous vehicle system. Identify and describe the Performance Measure, Environment, Actuators, and Sensors required for the agent. (BTL 4, 8 Marks)

1. The PEAS Architectural Framework: In AI system engineering, an intelligent agent cannot be designed until its **Task Environment** is formally specified. The **PEAS Framework** provides the rigorous 4-dimensional specification:

- **P — Performance Measure:** The objective external criterion used to evaluate how successfully the agent achieves its intended task. Must be defined externally by system designers.
- **E — Environment:** The external operational world, physical context, and dynamic obstacles within which the agent operates.
- **A — Actuators:** The physical or software mechanisms through which the agent exerts control and delivers actions to change the environment.
- **S — Sensors:** The perceptual hardware devices and software interfaces through which the agent receives percepts from the environment.

2. Detailed PEAS Specification for an Autonomous Vehicle System:

PEAS Dimension	Comprehensive Engineering Specification for Autonomous Vehicle
Performance Measure (P)	• Safety: Zero collisions, pedestrian avoidance, adherence to traffic laws and speed limits. • Trip Efficiency: Minimizing travel time, avoiding congested bottlenecks.

PEAS Dimension	Comprehensive Engineering Specification for Autonomous Vehicle
	• Passenger Comfort: Smooth acceleration/deceleration, jerk minimization, gradual lane changes. • Economic / Environmental Cost: Fuel / battery energy optimization, minimizing brake wear. • Legal Compliance: Obeying traffic signals, lane markings, and right-of-way rules.
Environment (E)	• Public city roads, interstate freeways, rural unmarked roads, intersections, bridges, tunnels. • Dynamic agents: Pedestrians, cyclists, erratic human drivers, emergency vehicles, stray animals. • Weather & lighting conditions: Rain, fog, snow, direct sun glare, night driving. • Physical road infrastructure: Traffic lights, stop signs, construction detours, potholes.
Actuators (A)	• Drive-by-Wire Steering System: Electric power steering motors for angle control. • Electronic Throttle Control: Accelerators for vehicle propulsion and cruise control. • Anti-lock Braking System (ABS): Hydraulic brake actuators for emergency/gradual stopping. • Transmission Controller: Gear shifting mechanisms (Drive, Reverse, Park, Neutral). • Signaling Subsystems: Turn indicator lights, headlights, horn, infotainment display screen.
Sensors (S)	• LiDAR Sensors: Roof-mounted 360-degree laser scanners providing high-density 3D point clouds. • Optical Cameras: High-definition front/rear/side vision cameras for lane detection, sign reading. • RADAR Sensors: Millimeter-wave radar for velocity estimation and all-weather obstacle detection. • Ultrasonic Sonar Sensors: Short-range proximity sensors for parking and blind-spot detection. • GPS & IMU: Global Positioning System receivers combined with Inertial Measurement Units. • Wheel Speed Encoders: Odometry sensors measuring exact tire revolutions and wheel slip.

3. Secondary PEAS Specification: Medical Diagnosis Expert System:

- **Performance Measure:** Diagnostic accuracy (%), sensitivity, specificity), patient survival rate, minimization of unnecessary surgical/biopsy costs, treatment recovery time, absence of adverse drug interactions.
- **Environment:** Hospital wards, intensive care units (ICU), pathology laboratories, diverse patient demographics.
- **Actuators:** Diagnostic report generation screens, medication dosage recommendations, electronic prescription generation, automatic alerts to attending physicians.
- **Sensors:** Electronic Health Records (EHR) text input, lab blood test analyzers, radiology imaging scans (CT, MRI, X-ray), patient vital monitors (ECG, pulse oximeter, blood pressure cuffs).

4. Environmental Properties Classification for Autonomous Vehicle System:

- **Partially Observable:** Sensors cannot see behind large trucks or through dense fog.
- **Stochastic:** Next traffic states cannot be predicted with 100% certainty due to human unpredictability.
- **Sequential:** Current braking and steering decisions directly affect future vehicle trajectories.
- **Dynamic:** Traffic and pedestrians move continuously while the agent computes its next action.
- **Continuous:** Speed, steering angles, and spatial coordinates vary continuously over time.
- **Multi-Agent (Competitive + Cooperative):** Operates alongside other drivers and pedestrians.

Source: Russell & Norvig (AIMA 4th ed), Ch 2, p. 40-48

Topic 1.4: Real-World Applications of AI & Major Challenges / Ethical Issues

GROUP:

Consolidated Questions Covered:

- **Q5:** Discuss the applications of Artificial Intelligence in different domains. Also explain the major challenges associated with AI. (BTL 3, 7 Marks)

1. Cross-Domain Applications of Artificial Intelligence:

Domain	Core AI Technologies Used	Industrial Impact & Use Cases
Healthcare & Medicine	Deep Learning CNNs, Transformers, Knowledge Graphs.	Early cancer detection in histopathology, drug discovery molecule screening, robotic microsurgery assistance (da Vinci system).
Autonomous Systems & Transportation	Computer Vision, Sensor Fusion, Deep Reinforcement Learning.	Self-driving automobiles (Waymo, Tesla FSD), automated drone delivery, intelligent traffic signal timing.
Banking, Finance & Fin-tech	Ensemble Decision Trees, Anomaly Detection, Time-Series LSTMs.	Real-time credit card fraud detection, algorithmic high-frequency trading, automated creditworthiness scoring.
Natural Language & Customer Support	Large Language Models (LLMs), RAG pipelines, Speech Recognition.	Automated enterprise customer service chatbots, multi-language real-time translation, automated legal contract review.
Agriculture & Smart Farming	Hyperspectral Drone Vision, IoT edge ML models.	Precision weed eradication using targeted spraying robots, crop disease classification, automated yield prediction.
Smart Manufacturing (Industry 4.0)	Predictive Analytics, Edge Vision Inspection.	Predictive maintenance for turbine bearings, automated optical quality control, robotic assembly line packing.

2. Major Challenges & Ethical Dilemmas Associated with AI:

1. Explainability & The “Black-Box” Problem (eXplainable AI - XAI):

- Deep Neural Networks comprise hundreds of millions of non-linear parameters. While they achieve high predictive accuracy, their internal reasoning is opaque. In critical sectors (e.g., approving loan applications or criminal recidivism scoring), black-box decisions violate regulatory requirements (such as the EU GDPR “Right to Explanation”).

2. Data Bias, Algorithmic Fairness & Discrimination:

- Machine learning models inherit and amplify historical human biases present in training data (e.g., facial recognition exhibiting higher error rates for minority demographic groups; AI resume screening favoring specific genders).

3. Data Privacy, Surveillance & Security Vulnerabilities:

- Training frontier models requires massive datasets, raising severe concerns regarding unauthorized scraping of personal data, non-consensual biometric tracking, and vulnerability to **Adversarial Attacks** (subtle pixel noise causing a vision classifier to misidentify a stop sign as a speed limit sign).

4. Safety, Robustness & Hallucinations:

- Generative AI models generate plausible-sounding but factually fabricated outputs (**Hallucinations**). Autonomous agents can suffer distribution shifts and cause fatal accidents under out-of-distribution environmental anomalies.

5. High Environmental & Energy Footprint:

- Training state-of-the-art transformer models requires megawatts of electricity and thousands of high-performance GPU clusters, creating significant carbon footprints and water cooling consumption.

Source: Russell & Norvig (AIMA 4th ed), Ch 1 & 27, p. 20-32, 1010-1035

Topic 1.5: Problem Formulation & State-Space Representation (Warehouse Robot Problem)

GROUP:

Consolidated Questions Covered:

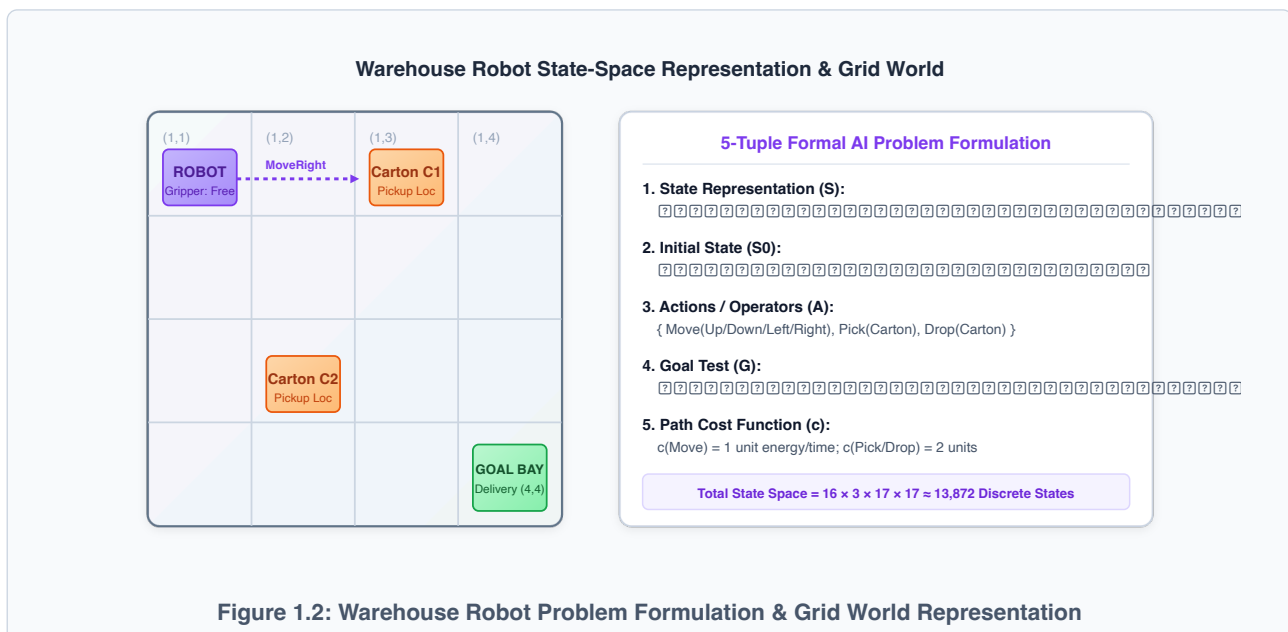
- **Q7:** Apply the concepts of problem formulation and state-space representation to a warehouse robot tasked with shifting cartons. Define initial state, goal state, actions, state space, and path cost. (BTL 3, 8 Marks)
- **Q11:** Explain Problem Formulation and State Space Representation in Artificial Intelligence. Describe the major components of a well-formulated AI problem using the scenario of a robot used in a warehouse to shift cartons. (BTL 3, 7 Marks)

1. Academic Concept of Problem Formulation & State-Space Representation: In Artificial Intelligence, **Problem Formulation** is the formal process of deciding what actions and states to consider, given a goal. It abstracts the infinite complexity of the physical real world into a discrete mathematical model that a search algorithm can systematically explore.

A **State Space** is the set of all valid configurations reachable from the initial state through any sequence of valid actions. It forms a directed graph (V, E) where nodes V represent discrete states and edges E represent actions.

2. The 5 Formal Components of a Well-Formulated AI Problem: According to Russell & Norvig, every well-defined AI problem is specified as a **5-Tuple**: $(S_0, A(s), \text{Result}(s, a), \text{GoalTest}(s), c(s, a, s'))$:

1. **Initial State (S_0):** The exact starting configuration of the agent and environment.
2. **Actions / Operators ($A(s)$):** The set of legal actions executable by the agent from a given state s .
3. **Transition Model ($\text{Result}(s, a)$):** The formal specification of the successor state s' resulting from executing action a in state s .
4. **Goal Test ($\text{GoalTest}(s)$):** A boolean function evaluating whether a given state s satisfies the objective condition.
5. **Path Cost Function ($c(s, a, s')$):** A function assigning a numeric cost to taking action a from state s to state s' . The total path cost is the sum of individual step costs.



3. Formal Problem Formulation: Warehouse Robot Shifting Cartons:

• Scenario Context:

- A warehouse is modeled as an $M \times N$ discrete 2D grid (4×4 grid, coordinates $(x, y) \in [1, 4] \times [1, 4]$).

- The robot has a single-item mechanical gripper that can be either Free or holding a carton ($\text{Holding}(C_i)$).
- Two distinct cartons, C_1 and C_2 , start at designated pickup locations and must both be shifted to the designated **Loading Bay** at coordinate $(4, 4)$.

• **Formal 5-Tuple Specification:**

1. **State Representation (S):**

$$S = \langle \text{Robot_Loc}, \text{Gripper_Status}, \text{Loc}(C_1), \text{Loc}(C_2) \rangle$$

Where:

- $\text{Robot_Loc} \in \{(x, y) \mid 1 \leq x \leq 4, 1 \leq y \leq 4\}$ (16 possible positions).
- $\text{Gripper_Status} \in \{\text{None}, C_1, C_2\}$.
- $\text{Loc}(C_1) \in \{(x, y) \mid 1 \leq x \leq 4, 1 \leq y \leq 4\} \cup \{\text{In_Gripper}\}$.
- $\text{Loc}(C_2) \in \{(x, y) \mid 1 \leq x \leq 4, 1 \leq y \leq 4\} \cup \{\text{In_Gripper}\}$.

2. **Initial State (S_0):**

$$S_0 = \langle (1, 1), \text{None}, (1, 3), (3, 2) \rangle$$

(Robot at $(1, 1)$ with free gripper; C_1 at $(1, 3)$; C_2 at $(3, 2)$).

3. **Actions / Operators ($A(s)$):**

- $\text{Move}(d)$ where $d \in \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$ (Precondition: Target grid cell is within boundaries $[1, 4] \times [1, 4]$ and not blocked).
- $\text{Pick}(C_i)$ (Precondition: $\text{Robot_Loc} = \text{Loc}(C_i)$ AND $\text{Gripper_Status} = \text{None}$).
- $\text{Drop}(C_i)$ (Precondition: $\text{Gripper_Status} = C_i$).

4. **Transition Model ($\text{Result}(s, a)$):**

- If action is $\text{Move}(\text{Right})$ from state $\langle (x, y), g, l_1, l_2 \rangle$:
 - If $g = \text{None}$: Successor state is $\langle (x+1, y), \text{None}, l_1, l_2 \rangle$.
 - If $g = C_1$: Successor state is $\langle (x+1, y), C_1, (x+1, y), l_2 \rangle$ (carton moves with robot).
- If action is $\text{Pick}(C_1)$ at (x, y) : Successor state is $\langle (x, y), C_1, \text{In_Gripper}, l_2 \rangle$.
- If action is $\text{Drop}(C_1)$ at (x, y) : Successor state is $\langle (x, y), \text{None}, (x, y), l_2 \rangle$.

5. **Goal Test ($\text{GoalTest}(s)$):**

$$\text{GoalTest}(s) = \text{True} \iff (\text{Loc}(C_1) = (4, 4) \wedge \text{Loc}(C_2) = (4, 4) \wedge \text{Gripper_Status} = \text{None})$$

6. **Path Cost Function ($c(s, a, s')$):**

- $c(\text{Move}) = 1$ unit (energy / time consumed per step).
- $c(\text{Pick}) = 2$ units (gripper alignment and pneumatic latching).
- $c(\text{Drop}) = 2$ units (precision placement and unlatching).
- **Optimization Goal:** Find an action sequence that shifts all cartons to $(4, 4)$ minimizing total cumulative cost:

$$\text{Total Cost} = \sum_{t=1}^{\{k\}} c(s_{\{t-1\}}, a_t, s_t)$$

Source: Russell & Norvig (AIMA 4th ed), Ch 3, p. 63-75

Unit 2: Problem Solving, Search & Knowledge Representation (CO302.2)

Topic 2.1: Search Strategies in AI: Uninformed Search (Uniform-Cost Search vs. Depth-First Search)

GROUP:

Consolidated Questions Covered:

- **Q10:** What are the different types of search strategies in AI? Explain Uniform-Cost Search (UCS). Compare UCS with Depth-First Search (DFS) based on search strategy, data structure, completeness, optimality, advantages, and limitations. (BTL 4, 7 Marks)

1. Classification of Search Strategies in AI: Search algorithms in Artificial Intelligence are systematically classified into two fundamental paradigms based on whether problem-specific domain knowledge is available:

- **1. Uninformed Search (Blind Search):** Algorithms explore the search space systematically using only the problem definition (state transitions and goal test) without any heuristic estimation of how close a state is to the goal.
 - Examples: **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, **Uniform-Cost Search (UCS)** / **Dijkstra**, **Depth-Limited Search (DLS)**, **Iterative Deepening Search (IDS)**.
- **2. Informed Search (Heuristic Search):** Algorithms utilize a problem-specific heuristic function $h(n)$ that estimates the remaining path cost from node n to the nearest goal state, directing exploration toward promising branches.
 - Examples: **Greedy Best-First Search**, **A* Search**, **Iterative-Deepening A* (IDA*)**, **Simplified Memory-Bounded A* (SMA*)**.

2. Detailed Working Principle of Uniform-Cost Search (UCS): Uniform-Cost Search (UCS) is an uninformed graph search algorithm that expands the node n with the lowest cumulative path cost $g(n)$ from the root initial state.

Algorithmic Mechanics:

- 1. Data Structure:** Maintains the **Frontier (Open List)** as a **Min-Priority Queue** keyed by path cost $g(n)$.
- 2. Expansion Step:** At each iteration, UCS pops the node n with minimum $g(n)$ from the priority queue.
- 3. Goal Test Timing:** Crucially, UCS performs the **Goal Test upon node removal/expansion**, NOT upon node generation. (Testing upon generation can return a suboptimal path if a cheaper path to the goal is discovered later).
- 4. Frontier Updating:** If a newly generated successor node n' is already present in the Frontier with a higher path cost, UCS updates its entry with the lower path cost (**Decrease-Key operation**).

3. Parameter-Based Comparison Table: Uniform-Cost Search (UCS) vs. Depth-First Search (DFS):

Parameter	Uniform-Cost Search (UCS)	Depth-First Search (DFS)
Search Strategy	Expands nodes in increasing order of cumulative path cost $g(n)$ (contours of equal cost radiate outward).	Expands the deepest unexpanded node in the current search frontier first (plunges down one branch).
Primary Data Structure	Min-Priority Queue (ordered ascending by path cost $g(n)$).	LIFO Stack (Last-In, First-Out) or recursive function call stack.
Completeness	Complete if all step costs $\epsilon > 0$ and branching factor b is finite.	Incomplete in infinite-depth spaces or spaces with cycles (can get trapped in infinite loops).
Optimality	Guaranteed Optimal ; always returns the path with minimal total cost C^* .	Not Optimal ; can return a very long/expensive path simply because it found it first.
Time Complexity	$O(b^{1+\lceil C^*/\epsilon \rceil})$, where C^* is optimal cost and ϵ is minimum step cost.	$O(b^m)$, where m is maximum depth of the state space ($m \gg d$).
Space Complexity	$O(b^{1+\lceil C^*/\epsilon \rceil})$ (high memory usage; stores all frontier nodes in RAM).	$O(b \cdot m)$ (Extremely modest linear memory; stores only single active branch).

Parameter	Uniform-Cost Search (UCS)	Depth-First Search (DFS)
Key Advantages	Guarantees shortest/cheapest path for arbitrary positive edge costs.	Minimal memory consumption; finds deep solutions quickly if many paths exist.
Key Limitations	Can consume excessive memory and time exploring many low-cost branches in all directions.	Risk of non-termination in infinite search trees; no guarantee of solution quality.

Source: Russell & Norvig (AIMA 4th ed), Ch 3, p. 75-88

Topic 2.2: Informed Search & Heuristic Functions: Greedy Best-First Search vs. A* Search

GROUP:

Consolidated Questions Covered:

- **Q8:** Apply the concept of heuristic functions to explain how Greedy Best-First Search and A* Search select nodes during informed search. Illustrate the working principle of both algorithms with a suitable example. (BTL 3, 7 Marks)
- **Q12:** Explain the role of a heuristic function in informed search. Describe the working principle of Greedy Best-First Search and A* Search. (BTL 3, 7 Marks)

1. The Role and Properties of Heuristic Functions ($h(n)$): A **Heuristic Function** $h(n)$ is a problem-specific estimation function that calculates the estimated cost of the cheapest path from state at node n to a goal state:

$$h : \text{Nodes} \rightarrow \mathbb{R}^+ \cup \{0\}$$

Properties for Optimality in A* Search:

1. **Admissibility (Admissible Heuristic):** A heuristic $h(n)$ is admissible if it **never overestimates** the true cost to reach the nearest goal ($h(n) \leq h^*(n)$, where $h^*(n)$ is true cost). It is optimistic.
2. **Consistency / Monotonicity (Consistent Heuristic):** For every node n and every successor n' generated by action a with step cost $c(n, a, n')$:

$$h(n) \leq c(n, a, n') + h(n')$$

Consistency satisfies the triangle inequality and guarantees that $f(n)$ is non-decreasing along any path, eliminating the need to reopen closed nodes.

2. Node Evaluation Function Comparison:

Greedy Best-First Search:

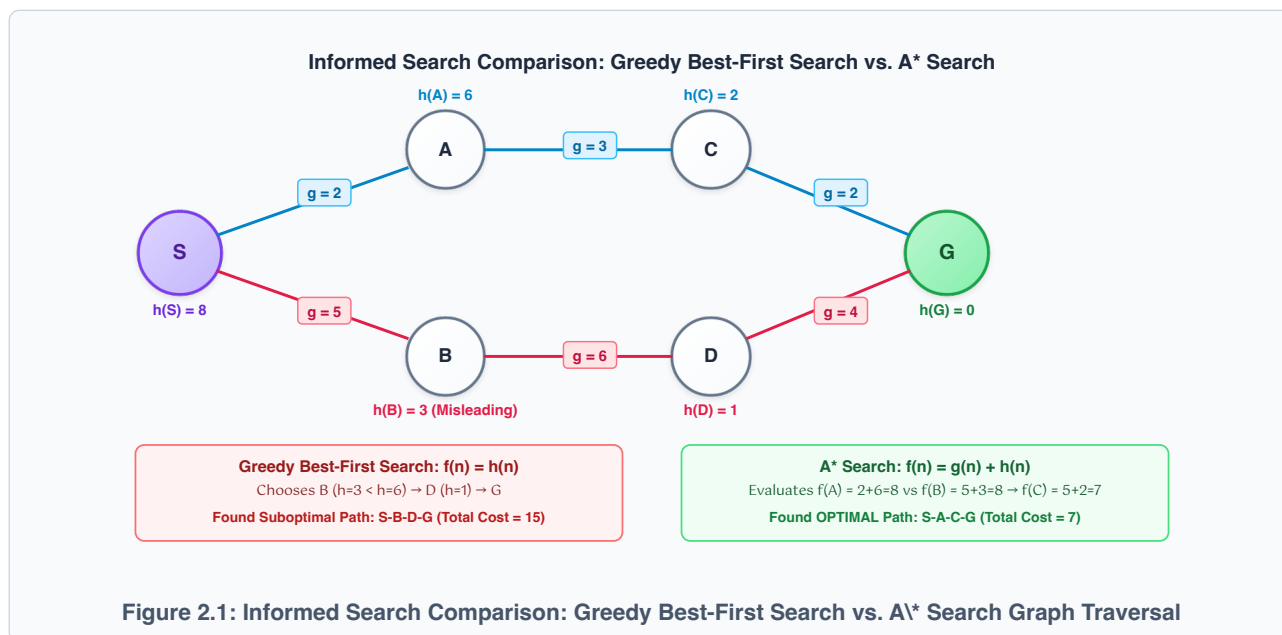
$$f(n) = h(n)$$

Selects the node that appears to be closest to the goal according to the heuristic alone, completely ignoring the cost $g(n)$ accumulated so far.

A* Search:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the exact known cost from start node to node n , and $h(n)$ is the estimated cost from node n to goal. Thus, $f(n)$ represents the **estimated total cost of the cheapest solution path through node n** .



3. Step-by-Step Graph Traversal Example: Consider the search graph in Figure 2.1 with Start Node S and Goal Node G :

- Nodes with Heuristic values: $h(S) = 8, h(A) = 6, h(B) = 3, h(C) = 2, h(D) = 1, h(G) = 0$.
- Edge Path Costs (g):
 - $S \rightarrow A$ (cost 2), $S \rightarrow B$ (cost 5).
 - $A \rightarrow C$ (cost 3), $C \rightarrow G$ (cost 2).
 - $B \rightarrow D$ (cost 6), $D \rightarrow G$ (cost 4).

Execution Trace 1: Greedy Best-First Search ($f(n) = h(n)$)

1. **Initial State:** Expand S . Successors: A with $h(A) = 6$, B with $h(B) = 3$.
2. **Greedy Choice:** Selects B because $h(B) = 3 < h(A) = 6$ (misled by heuristic).
3. **Expand B :** Successor is D with $h(D) = 1$.
4. **Expand D :** Successor is G with $h(G) = 0$. Goal reached!
5. **Resulting Path:** $S \rightarrow B \rightarrow D \rightarrow G$ with Total Cost = $5 + 6 + 4 = 15$. (Suboptimal!!).

Execution Trace 2: A* Search ($f(n) = g(n) + h(n)$)

1. **Initial State:** Expand S ($g = 0$). Successors:
 - Node A : $g(A) = 2, h(A) = 6 \Rightarrow f(A) = 2 + 6 = 8$.
 - Node B : $g(B) = 5, h(B) = 3 \Rightarrow f(B) = 5 + 3 = 8$.
 - (Tie-break selects A).
2. **Expand A :** Successor C :
 - Node C : $g(C) = 2 + 3 = 5, h(C) = 2 \Rightarrow f(C) = 5 + 2 = 7$.
3. **Frontier State:** $\{C(f=7), B(f=8)\}$. Lowest f is node C ($f=7$).
4. **Expand C :** Successor G :
 - Node G : $g(G) = 5 + 2 = 7, h(G) = 0 \Rightarrow f(G) = 7 + 0 = 7$.
5. **Frontier State:** $\{G(f=7), B(f=8)\}$. Lowest f is node G .
6. **Expand G :** Goal tested on expansion $\Rightarrow$ Goal Reached!
7. **Resulting Path:** $S \rightarrow A \rightarrow C \rightarrow G$ with Total Cost = $2 + 3 + 2 = 7$. (Optimal!!).

4. Parameter-Based Comparison Table:

Parameter	Greedy Best-First Search	A* Search
Evaluation Function	$f(n) = h(n)$ (heuristic estimation only).	$f(n) = g(n) + h(n)$ (past cost + future estimate).
Optimality	Not Optimal ; can be misled by low heuristic estimates into expensive paths.	Optimal (if $h(n)$ is admissible for trees / consistent for graphs).

Parameter	Greedy Best-First Search	A* Search
Completeness	Incomplete in infinite graphs or with cycles (can loop indefinitely).	Complete (on finite graphs with positive step costs $\epsilon > 0$).
Time Complexity	$O(b^m)$ in worst case; good heuristics dramatically reduce search time.	$O(b^d)$ where error in heuristic is small; exponentially pruned.
Space Complexity	$O(b^m)$ (stores all generated nodes in memory).	$O(b^d)$ (keeps all nodes in memory; high memory footprint).
Sensitivity to $h(n)$	Extremely high; poor heuristics cause erratic exploration.	Robust; balanced by actual path cost $g(n)$ accumulation.

Source: Russell & Norvig (AIMA 4th ed), Ch 3, p. 88-105

Topic 2.3: Knowledge Representation: Propositional Logic vs. First-Order Predicate Logic

GROUP:

Consolidated Questions Covered:

- **Q13:** Describe Propositional Logic and Predicate Logic, and highlight the key differences between them with suitable examples. (BTL 3, 8 Marks)

1. Academic Overview of Propositional Logic (Boolean Logic / Zeroth-Order Logic): Propositional Logic (PL) is the simplest formal logic system where the fundamental building blocks are atomic propositional symbols (P, Q, R) that represent entire declarative statements capable of being either **True** or **False**.

- **Logical Connectives:** Negation ($\neg$), Conjunction ($\wedge$), Disjunction ($\vee$), Implication / Conditional ($\rightarrow$), Biconditional ($\leftrightarrow$).
- **Semantic Evaluation:** Evaluated via Truth Tables. An interpretation assigns a boolean value to each atomic proposition.
- **Limitation:** Lacks internal structure; cannot express objects, properties of objects, or relationships among collections. (e.g., to state "All students are smart" in PL requires writing separate propositions $P_1, P_2, \dots, P_{\{1000\}}$ for each student).

2. Academic Overview of First-Order Predicate Logic (FOL): First-Order Predicate Logic (FOL / FOPL) is a far more expressive formal language that models the world in terms of:

1. **Objects:** Nouns representing individual entities (Ashley, Pune, 42, Carton₁).
2. **Predicates / Relations:** Properties or relations holding between objects (IsStudent(x), Greater(x, y), IsWet(Ground)).
3. **Functions:** Mappings returning an object given an input argument (MotherOf(x), LocationOf(C_1)).
4. **Quantifiers:**
 - **Universal Quantifier ($\forall x$):** "For all x / for every x ".
 - **Existential Quantifier ($\exists x$):** "There exists at least one x ".

3. Illustrative Translation Examples:

Natural Language Statement	Propositional Logic (PL)	First-Order Predicate Logic (FOL)
"All humans are mortal."	Cannot generalize; requires P ("Socrates is mortal") $\wedge Q$ ("Plato is mortal").	$\forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$
"Every student loves AI."	Atomic proposition S (truth value only).	$\forall x(\text{Student}(x) \rightarrow \text{Loves}(x, \text{AI}))$
"Some birds cannot fly."	Atomic proposition B .	$\exists x(\text{Bird}(x) \wedge \neg \text{CanFly}(x))$
"If it rains, the ground is wet."	$P \rightarrow Q$ (where $P = \text{Raining}$, $Q = \text{Ground is wet}$)	$\forall d(\text{Raining}(d) \rightarrow \text{IsWet}(\text{Ground}, d))$

4. Parameter-Based Comparison Table: Propositional Logic vs. Predicate Logic:

Parameter	Propositional Logic (PL)	First-Order Predicate Logic (FOL)
Ontological Commitment	Assumes the world contains binary facts that are either True or False.	Assumes the world contains distinct Objects, Relations, Properties, and Functions.
Epistemological Commitment	Agent believes a proposition is True, False, or Unknown.	Agent believes an assertion is True, False, or Unknown.
Expressive Power	Low / Restrictive; cannot quantify over collections or represent object attributes.	Very High; rich representation of generalized rules, quantified assertions, and multi-arity relations.
Quantification	No quantifiers supported ($\forall$ and $\exists$ do not exist in PL).	Fully supports Universal ($\forall$) and Existential ($\exists$) quantification.
Computational Complexity	Decidable; NP-Complete (Boolean Satisfiability / SAT problem).	Semi-Decidable; validity is provable, but non-entailment is undecidable in general.
Inference Algorithms	Truth Tables, Resolution, DPLL Algorithm, WalkSAT.	Unification, Generalized Modus Ponens, First-Order Resolution, Skolemization.

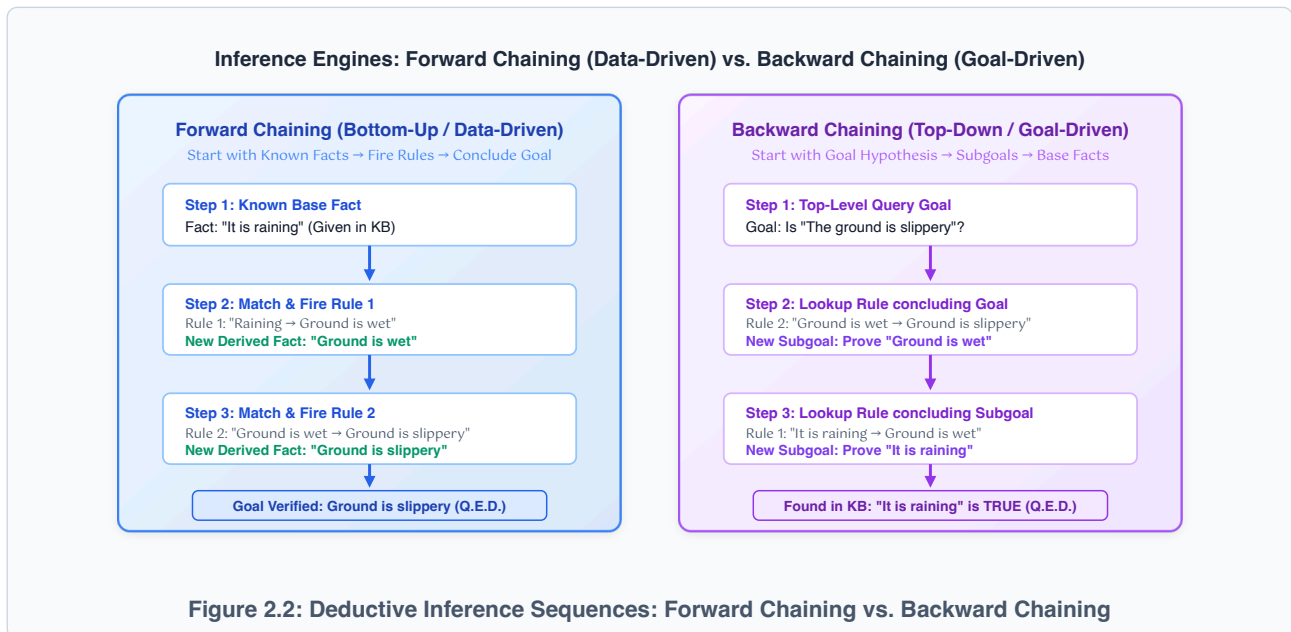
Source: Russell & Norvig (AIMA 4th ed), Ch 7 & 8, p. 210-250, 270-300

Topic 2.4: Inference Engines: Forward Chaining vs. Backward Chaining Deductive Reasoning**GROUP:****Consolidated Questions Covered:**

- **Q9:** Consider the knowledge base: Fact: "It is raining"; Rule 1: "If it is raining, then the ground is wet"; Rule 2: "If the ground is wet, then the ground is slippery". Analyze how Forward Chaining and Backward Chaining derive "The ground is slippery". Compare reasoning sequences. (BTL 4, 8 Marks)
- **Q14:** Consider the same KB. Suppose fact "It is raining" is known. Explain how Forward Chaining reaches conclusion "The ground is slippery", and how Backward Chaining works when goal is "The ground is slippery". Show sequence of reasoning. (BTL 4, 8 Marks)

1. Formal Knowledge Base (KB) Formulation: Let the given facts and rules be represented symbolically:

- **Known Base Fact (F_1):** R (where R = It is raining)
- **Rule 1 (R_1):** $R \rightarrow W$ (where W = The ground is wet)
- **Rule 2 (R_2):** $W \rightarrow S$ (where S = The ground is slippery)
- **Query / Target Goal:** Prove S (The ground is slippery).



2. Forward Chaining (Data-Driven / Bottom-Up Reasoning Sequence): Forward Chaining starts with the known atomic facts in the Knowledge Base and applies the inference rule **Modus Ponens** ($\frac{\alpha}{\alpha \rightarrow \beta}(\beta)$) iteratively in a forward direction to extract new facts until the target goal is derived.

Step-by-Step Reasoning Trace:

1. **Initial State:**
 - Current Known Facts Set: $Facts = \{R\}$
 - Goal to prove: S
2. **Iteration 1:**
 - Scan rule base against $Facts$: Rule $R_1(R \rightarrow W)$ has its premise R satisfied ($R \in Facts$).
 - **Fire Rule R_1 :** By Modus Ponens, deduce conclusion W (The ground is wet).
 - Update Known Facts: $Facts = \{R, W\}$.
3. **Iteration 2:**
 - Scan rule base against updated $Facts$: Rule $R_2(W \rightarrow S)$ has its premise W satisfied ($W \in Facts$).
 - **Fire Rule R_2 :** By Modus Ponens, deduce conclusion S (The ground is slippery).
 - Update Known Facts: $Facts = \{R, W, S\}$.
4. **Termination & Verification:**
 - The derived fact S matches the Query Goal.
 - **Conclusion:** Forward Chaining successfully derives S in 2 deduction steps:

$$R \Rightarrow (R \rightarrow W) \Rightarrow W \Rightarrow (W \rightarrow S) \Rightarrow S(\text{Q.E.D.})$$

3. Backward Chaining (Goal-Driven / Top-Down Reasoning Sequence): Backward Chaining starts with the Query Goal hypothesis and works backward, finding rules whose conclusions match the current goal, and establishing their premises as new subgoals recursively until base facts in the KB are reached.

Step-by-Step Reasoning Trace:

1. **Initial State:**
 - Top-Level Goal Hypothesis: $G_0 = S$ (The ground is slippery).
 - Check if $S \in Facts$: No.
2. **Step 1 (Subgoal Generation via R_2):**
 - Search KB for rules concluding S : Finds Rule $R_2(W \rightarrow S)$.
 - To prove S , the engine must establish the antecedent premise W as a **New Subgoal** ($G_1 = W$).
3. **Step 2 (Subgoal Generation via R_1):**
 - Check if $W \in Facts$: No.
 - Search KB for rules concluding W : Finds Rule $R_1(R \rightarrow W)$.
 - To prove W , the engine must establish the antecedent premise R as a **New Subgoal** ($G_2 = R$).
4. **Step 3 (Base Fact Verification):**

- Check if $R \in \text{Facts}$: **YES!** R (It is raining) is an established primitive base fact in the KB.

5. **Step 4 (Unwinding / Back-Propagation of Truth):**

- Since R is True $\implies$ Rule R_1 fires $\implies$ Subgoal W is verified True.
- Since W is True $\implies$ Rule R_2 fires $\implies$ Query Goal S is verified True.
- **Conclusion:** Backward Chaining confirms S via goal hierarchy:

$$S \Leftarrow [W \text{ via } R_2] \Leftarrow [R \text{ via } R_1] \Leftarrow [\text{Base Fact } R \text{ in KB}] (\text{Q.E.D.})$$

4. **Parameter-Based Comparison Table: Forward Chaining vs. Backward Chaining:**

Parameter	Forward Chaining (Data-Driven)	Backward Chaining (Goal-Driven)
Reasoning Direction	Bottom-Up: Starts from initial data/facts and moves forward toward conclusions.	Top-Down: Starts from target goal hypothesis and works backward to supporting facts.
Driving Mechanism	Data-Driven: Triggered when new facts are asserted into the working memory.	Goal-Driven: Triggered when a user/system poses a specific query or hypothesis to test.
Search Space Exploration	Explores all reachable inferences; may generate many irrelevant facts.	Focused search; explores only rules and facts directly relevant to the target goal.
Inference Primitive	Modus Ponens forward rule matching $(\frac{\alpha}{\alpha \rightarrow \beta}(\beta))$.	Goal reduction, AND-OR tree resolution, unification backtracking.
Computational Efficiency	Can be slow and compute-heavy if many rules match unrelated facts.	Highly efficient for answering specific single-point queries in large knowledge bases.
Typical Application Domains	Real-time monitoring, process control systems, data mining, automated synthesis.	Medical diagnostic expert systems (MYCIN), automated theorem proving, Prolog engines.

Source: Russell & Norvig (AIMA 4th ed), Ch 7 & 9, p. 235-245, 320-345